CAWJ - Assignment -1

Q1- Explain how Java achieves platform independence through the concepts of Bytecode and JVM.

A- Java is a platform independant language as the same java program can run on different operating systems without modifying the source code

→ achived via Byte code and JVM (Java Virtual Machine) as follows;

- Java source code/ program written in a .java source file
- The Java Compiler (Javac) compiles the source code into byte code stored in a ".class" file

XYZ.java → javac compiler → XYZ.class (Byte code) → not specific to any OS or processor.

- byte code is not directly understood by a computer it is executed by the JVM
- Each system has its own JVM implementation
  ↳ JVM converts the same bytecodes according to the platform

→ Reason for platform independance :
↳ Byte code is platform independent.

Q-2- What do you mean by dynamic binding? How is it useful to Object Oriented Programming?

A→ Dynamic binding is the process in which the method to be executed is decided at runtime. instead of compile time. also called "late binding" or runtime "binding". It mainly occurs when method over ridding used with inheritance and super class refers to subclass object.

```
eg- class Animal {
      void sound(){
          System.out.println("Animal makes sound")
      }
    }

    class Dog extends Animals
    { void sound()
      { System.out.println
          ("Dog barks");
      }
    }
```

```
class Main ()
{  Animal a = new Dog();
   a.sound();
}
```

Output:
Dog barks.

→ the ref variable type "a" is Animal, but the actual obj is Dog type.
at runtime java defines/determines that the Dog version of sound() should be executed.

Useful uses of dynamic binding in OOPs:-
↳ Supports polymorphism. (runtime) [same method can be diff for diff objects.]
↳ Supports method overriding:
↳ child class can provide their own implementation..
↳ Makes program more flexible & extensible:
↳ appropriate method selected according to actual obj:
↳ Improves code reusability.:
↳ common parent reference can be used for different child obj
↳ Reduces dependency on specific classes.
↳ Makes it easier to add new subclasses without changing existing code much.

Q-3 Discuss the difference in Programing using Procedural Languages and Object-Oriented Languages.

A→

Procedural Programing	Object-Oriented Programming
→ program organized around functions / procedures.	→ Program organized around objects and classes.
→ focuses mainly on what the program should do	→ focuses on objects & their behaviours.
→ Data & fn" are generally separated	→ Data & methods combined inside object.
→ Uses a top-down approach	→ Uses a bottom-up approach.
→ Code reuse achieved via fn"	→ Code reuse supported through: Inheritence, composition & polymorphism.

Procedural Programing	Object-Oriented Programming
→ Data maybe accessible to several fx"	→ Data can be protected via encapsulation & access modifiers.
→ less suitable for large & complex system.	→ well suited for large and complex applications.
→ [redacted]	
→ Programs can be more difficult to maintain, modify, find/fix error as they become larger.	→ Programs are easier to understand, maintain & modify, and for finding & fixing errors.

Q-4 Describe Inheritance as applied to Object Oriented Programing?

A→ Inheritance is an important feature of OOPs in which one class acquires properties & behaviour of another class

↳ existing class = super class / parent / base class

↳ new class = sub class / child / derived class

→ In Java inheritance is implemented using the "extend" keyword.

→ main purpose of inheritance is to reuse existing code & create a relationship between classes.

→ Types of inheritance

↳ Single Inheritance

↳ multilevel Inheritance

↳ Hierarchical Inheritance

↳ Multiple inheritance. (can be achieved using interfaces)

Eg:-

```
class Animal
{ void eat()
  { System.out.println("Animal eats");}
}
```

```
class Dog extends Animal {
{ void bark() {
  { System.out.println("Dog barks");}
}
```

* Here dog is a subclass that inherits eat() method from Animal (super class), while also having its own method bark()

Q-5 What do you mean by Just In Time (JIT compilation)?

A→ Just In Time (JIT) compilation is a feature of the JVM that improves the performance of Java programs by converting frequently executed byte code into native machine code at runtime.

flow → Java source code → javac → Bytecode → JVM (initially interprets byte code) → Machine code.

★ Benifits of JIT:

↳ Improves programs execution speed by combining interpretation with runtime compilation.

↳ reduces frequent interpretations of frequently executed code.

↳ JIT compilations helps Java achive better performance while maintaining platform independence through JVM based byte code execution.

Q- 6. Discuss various features of Java.

A→ → Java has a clear syntax and removes complex features, machine programing easier for developers.

→ Object oriented:

↳ Java organises programs using classes & objects and supports → encapsulation
→ inheritance
→ polymorphism.
→ abstraction.

→ Platform independant:

↳ Java source code compiled into platform independant bytecode, which can be executed by JVM available for different operating systems.

→ Portable:

↳ program can be transfered between different platforms. and source-code modification.

→ Robust:

Java provides → strong type checking.
↳ Exception handling.
↳ Auto memory allocation
↳ Garbage collection.

classmate
Date
Page

→ Multi threaded
provides built-in support for multithreading.

→ High Performance (using JIT compiler